

Performance

(reference: text Sections 1.6, 1.9, 1.11)

“How fast does my code run”, but not quite as simple as that owing to parallelism (might get speedup by doing more things simultaneously rather than any one thing quicker, e.g. massive parallelism in Google’s implementation of search ...)

two system-level metrics are **response time** and **throughput** (e.g. for Google search, how long does it take to get a response to a search request? at what rate can the system process requests?)

overall system performance can depend on more than just the processor (e.g., secondary storage devices, network links, ...)

however, we will focus for now just on the metric of **CPU execution time** ... i.e., how long it takes a single “CPU” (processor core) to execute some sequence of instructions

CPU execution time as the product of 3 factors

CPU execution time = (instruction count) x (clock cycle time) x CPI

instruction count: the number of machine language instructions executed

depends on:

- high-level language program
- compiler
- “instruction set architecture” (ISA) (how rich is the instruction set? One system might have a powerful machine language instruction that can do a complex operation (e.g., a vector operation) that takes a sequence of instructions on another system ... historically important distinction between **RISC** – “reduced instruction set computer” and **CISC** – “complex instruction set computer”)

clock cycle time: processors rely on a clocking signal to time their operations, the “clock cycle time” is the time between clock “ticks”

- **each step of instruction execution requires one clock cycle to complete**
- very short ... commonly measured in nanoseconds (one nanosecond is one billionth of a second, i.e. 10^{-9} seconds), or in picoseconds

depends on:

- architecture implementation

clock cycle time = $1 / \text{clock rate}$ (be careful re: units !!!)

- clock rate commonly measured in GHz (billions of cycles per second)
- e.g. if clock rate is 2 GHz, clock cycle time is 0.5 nanoseconds (= 500 picoseconds)

in the past, clock cycle time of a processor was fixed, but now often can be dynamically varied (and so it's actually the *average* clock cycle time that is a factor in the CPU execution time)

CPI (“cycles per instruction”): to simplify things, we will assume for now that instructions are executed one-by-one, without pipelining (later in the course we’ll see how idea can be generalized)

- CPI without any subscript is used for the **average number of clock cycles required per instruction executed**
- different types (classes) of instructions can have different average execution times, use CPI_c for the average number of clock cycles required for an instruction of class c

for example, suppose we have two types of instructions A and B; $CPI_A = 5$, $CPI_B = 1$; what is the CPI?

Is it $(5+1)/2 = 3$??

CPI can be expressed as a **weighted average**, where the weight for an instruction class c is the fraction of the executed instructions that are class c instructions (note that program dependent !!)

$$- \text{CPI} = \sum f_c \times \text{CPI}_c$$

In previous example, suppose 200 million type A instructions were executed, 300 million type B ... now we know enough to calculate the CPI (what is it?)

Another formula for CPI:

$$\frac{\text{total \# clock cycles required for program execution}}{\text{total \# machine language instructions executed}}$$

try for above example, should get the same answer as with weighted average formula

CPI depends on:

- ISA
- architecture implementation
- high-level language program
- compiler

CPU execution time = (instruction count) x (clock cycle time) x CPI

e.g. suppose 4 GHz clock rate, 800 million instructions, CPI = 1.5

CPU execution time = $800 \times 10^6 \times 1.5 \times (1/4 \times 10^{-9})$
= ?? seconds

example problem:

two types of instructions A and B; $CPI_A = 5$, $CPI_B = 1$;
40% of executed instructions are of type A, 60% are of type B

if new compiler reduced # class B instructions executed by 50%, by what factor would CPU execution time decrease?

solution:

- let C denote original instruction count
- original # class A instructions executed = $0.4C$
- original # class B instructions executed = $0.6C$
- original CPI = ??
- new # class B instructions executed = $0.3C$
- # class A instructions executed is unchanged (= $0.4C$)
- new instruction count = $0.4C + 0.3C = 0.7C$
- **new CPI** = (new fraction of instructions executed that are of type A) $\times 5$ + (new fraction of instructions executed that are of type B) $\times 1$

and so **ratio of new execution time to old execution time** is given by:

$0.7C \times \text{new CPI} \times \text{clock cycle time}$

$C \times \text{original CPI} \times \text{clock cycle time}$

$= (0.7 \times \text{new CPI}) / \text{original CPI}$

$= ??$

Amdahl's law: the performance improvement possible with a given system improvement is limited by the amount that the improved feature is used

new execution time = (portion of execution time affected by improvement)/(improvement factor) + portion of execution time unaffected

or alternatively, ratio of new to old execution time = (fraction affected)/(improvement factor) + fraction unaffected

in previous example problem:

fraction affected = (time spent executing type B instructions)/(total execution time)
= (0.6 x original instruction count x 1 x clock cycle time)/(original instruction count x original CPI x clock cycle time) = ??

fraction unaffected = 1 – fraction affected

improvement factor is 2

so, new/old = (fraction affected) / 2 + fraction unaffected = ??

Amdahl's law first used in the context of parallel computing and the limitations on speedup owing to unavoidably sequential portions of code (see [here](#) for an application to multicore processor design)